

The `impjax_toymodels` Wrapper: Reduced-Coordinate Sampling of IMP Systems with BlackJAX

Design document — `doc/design.tex`

Draft, 2026-08-13

Abstract

This document specifies, in full mathematical detail, how `wrapper_impjax.py` turns a built IMP/PMI system into a BlackJAX-samplable posterior and back again. It fixes: (1) the reduced coordinate system used for sampling (rigid-body quaternion/translation pairs plus flexible-bead coordinates), (2) the exact map between that reduced state and the flat per-particle array IMP’s JAX export expects, (3) SO(3)-compliant, detailed-balance-preserving proposal kernels for rotations, translations, and bead moves, (4) how these plug into the existing BlackJAX RMH/SMC wrappers in this repo without reimplementing any sampler, and (5) a GPU-aware I/O design for stat files and RMF3 trajectories. Every claim about IMP’s runtime behavior below (Section 3) was verified empirically against the `imp` conda environment, not assumed from documentation, since the JAX export layer in IMP is not otherwise documented.

Contents

1	Scope and non-negotiables	2
2	System state and notation	2
3	The IMP $\leftrightarrow$ JAX bridge	3
4	The expansion map Φ	4
5	Proposal kernels	4
5.1	Translation proposals (rigid-body and bead)	4
5.2	SO(3)-compliant rotation proposal	5
5.3	Composite, mode-masked proposal	5
6	Metropolis correctness	5
7	Wrapper pipeline	6
8	GPU-aware I/O	7
9	Fixed-schedule SMC	7
9.1	Particle population and tempered target	7
9.2	Initial population	8
9.3	Algorithm	8
9.4	Best-particle-only output	8
9.5	Untuned defaults	8

10	DEBUG-mode score verification	9
11	Timing methodology	9
12	Sampler comparison	9
13	Module layout	10
14	Assumptions and limitations	10

1 Scope and non-negotiables

Per `planning.md`, this repo must never reimplement a sampler: BlackJAX supplies the RMH/SMC/HMC kernels (already wrapped in `rmh_sampler.py` and `smc_base_sampler.py`); this repo’s only job is to supply BlackJAX with a JIT-able log-density function and a smart, $SO(3)$ -correct proposal. Concretely this document designs five small, single-purpose modules:

Module	Responsibility
<code>dof_layout.py</code>	Index bookkeeping: which particles are sampled, in what order, as what shape.
<code>state_sync.py</code>	IMP $\rightarrow$ reduced state (extract), reduced state $\rightarrow$ full IMP-shaped array (expand), reduced state $\rightarrow$ IMP (apply).
<code>proposals.py</code>	$SO(3)$ rotation, translation, and bead proposal kernels; composite proposal builder.
<code>wrapper_impjax.py</code>	Orchestration: build $\log \pi$, build proposal, call into <code>rmh_sampler.py</code> , sync results back.
<code>gpu_io.py</code>	Device-resident buffering, batched host flush, stat file + RMF3 writing.

None of these exceed the repo’s 200–300 line budget individually.

2 System state and notation

A built IMP system (`BuiltSystem` in `system_info.py`) exposes, via `rigid_bodies_and_beads()`, an ordered list of K rigid bodies and M flexible (non-rigid) beads. We fix this ordering once at wrapper construction time and never re-derive it mid-run, since every array in this design is positional.

Rigid body k . State is a pose

$$\theta_k = (q_k, t_k) \in \mathbb{S}^3 \times \mathbb{R}^3, \quad \|q_k\| = 1,$$

a unit quaternion $q_k = (w_k, x_k, y_k, z_k)$ (double-covering a rotation $R(q_k) \in SO(3)$) and a translation t_k of the rigid body’s reference frame. Each rigid body owns an ordered set of *member* particles $\mathcal{M}_k = \{1, \dots, n_k\}$ whose *local* (internal, frame-relative) coordinates $\ell_{k,i} \in \mathbb{R}^3$ are fixed for the life-time of the run (IMP: `RigidMember.get_internal_coordinates()`). A member’s global position is

$$g_{k,i}(\theta_k) = R(q_k) \ell_{k,i} + t_k. \tag{1}$$

Flexible bead m . State is simply its global coordinate $x_m \in \mathbb{R}^3$.

Reduced state. The full sampled state is the pytree

$$\theta = (\{q_k, t_k\}_{k=1}^K, \{x_m\}_{m=1}^M),$$

with dimension $7K + 3M$ (4 for each quaternion, 3 for each translation, 3 per bead). BlackJAX's samplers accept an arbitrary JAX pytree as the chain position (verified against `blackjax.mcmc.random_walk`, which types its position as `ArrayLikeTree` throughout), so the wrapper drives the walk directly on the structured dict $\theta = \{\text{quaternions} : (K, 4), \text{translations} : (K, 3), \text{bead_coords} : (M, 3)\}$ with no flattening required. `dof_layout.py` additionally defines a bijection $\text{flatten} : \theta \mapsto \mathbb{R}^{7K+3M}$ (and its inverse) used only for compact storage – the GPU IO buffer (Section 8) and stat-file rows – not to satisfy any BlackJAX requirement.

3 The IMP $\leftrightarrow$ JAX bridge

IMP exposes, on any `IMP.core.RestraintsScoringFunction`, a private method `_get_jax()` returning an object `ji` with two members:

- `ji.get_jax_model()` $\rightarrow$ `{"xyz": ndarray(N,3), "r": ndarray(N,)}` – **every** particle currently registered in the IMP Model (not just leaves: $N = 531$ for a 52-leaf, 2-domain toy dimer used in testing), indexed by *IMP particle index*. `r` is a static per-particle radius.
- `ji.score_func(model_dict)` $\rightarrow$ `scalar` – a pure, JAX-traceable function of the passed-in dict. Its value matches `RestraintsScoringFunction.evaluate(False)` on the corresponding IMP state exactly.

Assumption 1 (Verified empirically, not documented by IMP). *Three properties were confirmed by direct experiment (moving a particle and comparing `score_func` outputs; see repo history for the probe scripts) and are relied on throughout this design:*

1. `get_jax_model()["xyz"]` is a live view aliased to IMP's internal storage – mutating the IMP model changes previously-fetched dicts too. **Every module below explicitly copies this array** before treating it as an immutable JAX value.
2. `score_func` depends only on its argument, not on hidden IMP state – passing a hand-built `{"xyz": ..., "r": ...}` with the same shape reproduces IMP's own energy for that configuration. This is what makes the wrapper's log-density JIT-/grad-able.
3. Row i of `"xyz"` corresponds to IMP particle index i (0-based; the integer cast of a particle's `ParticleIndex`), not to leaf order or selection order.

Assumption 2 (Rigid-body transform convention). `IMP.algebra.Rotation3D(w, x, y, z)` implements the standard active rotation

$$R(q)v = v + 2(w(q_v \times v) + q_v \times (q_v \times v)), \quad q_v = (x, y, z),$$

and `RigidBody.get_reference_frame().get_transformation_to()` yields $(R(q_k), t_k)$ such that a member's global coordinate equals Eq. (1) exactly. Verified numerically to full float precision with a synthetic single-member rigid body and a 90° rotation.

Assumption 3 (Auxiliary, non-sampled rows). *Only 52 of 531 rows in the toy dimer correspond to sampled leaves (rigid-body members or flexible beads); the remainder are IMP bookkeeping particles (hierarchy/state/molecule/residue decorators) with no bearing on the restraints currently attached in this repo’s tests. These rows are frozen at their build-time template value for the lifetime of a sampling run. **This is a scoped assumption, not a general guarantee:** if a restraint is later added that reads a coarser-resolution representation not covered by the sampled leaves, that restraint’s contribution would silently use stale coordinates. Section 14 lists the regression test that guards this invariant for the restraints in use today, and flags it for re-verification whenever new restraint types are added.*

4 The expansion map Φ

`state_sync.py` builds $\Phi : \theta \mapsto \text{xyz} \in \mathbb{R}^{N \times 3}$:

$$\Phi(\theta)_j = \begin{cases} R(q_k) \ell_{k,i} + t_k & j \text{ is member } i \text{ of rigid body } k, \\ x_m & j \text{ is flexible bead } m, \\ \text{xyz}_j^{\text{template}} & \text{otherwise (Assumption 3).} \end{cases} \quad (2)$$

$\text{xyz}^{\text{template}}$ and r are captured *once*, as plain (copied) `numpy` arrays, when the wrapper is constructed — this is the one and only place IMP’s live-view aliasing (Assumption 1.1) is touched; everything downstream is ordinary immutable JAX. The member index sets $\{(k, i, j)\}$ and local coordinates $\{\ell_{k,i}\}$ are likewise captured once, from `RigidMember.get_internal_coordinates()`, and are treated as constants of the run (not resampled).

Log-posterior. IMP restraints are energies, satisfied at low values, so the (unnormalized) log-posterior used for Metropolis accept/reject is

$$\log \pi(\theta) = -\text{score_func}(\{\text{xyz} : \Phi(\theta), r\}) + \log p_0(\theta), \quad (3)$$

where p_0 is an optional prior over the reduced state (e.g. bounding-box or nuisance-parameter priors); it defaults to 0 (flat/improper) for the toy models in `test/`.

5 Proposal kernels

All kernels below are constructed so that the Metropolis ratio needs **no** Hastings correction term – i.e. proposal densities are symmetric, $q(\theta' | \theta) = q(\theta | \theta')$ – which is why `rmh_sampler.py` can plug a plain `blackjax.rmh(logdensity_fn, proposal)` kernel (its docstring already anticipates a caller-supplied `proposal(key, position) -> new_position`; we are that caller).

5.1 Translation proposals (rigid-body and bead)

Additive isotropic Gaussian:

$$t'_k = t_k + \sigma_t \varepsilon, \quad x'_m = x_m + \sigma_b \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_3). \quad (4)$$

Symmetric by construction ($\mathcal{N}(0, I_3)$ is symmetric about 0), so $q(t' | t) = q(t | t')$ trivially. Rigid bodies typically use a larger σ_t than individual beads, reflecting the different physical scales called out in `planning.md` (10s–100s of residues per rigid body vs. a handful per bead).

5.2 SO(3)-compliant rotation proposal

Naively perturbing quaternion components with i.i.d. Gaussian noise and renormalizing is *not* SO(3)-compliant: it distorts the invariant (Haar) measure on the rotation group and biases the proposal, which is exactly what `planning.md` rules out. Instead we perturb in the Lie algebra $\mathfrak{so}(3) \cong \mathbb{R}^3$ and map back via the exponential:

$$\omega \sim \mathcal{N}(0, \sigma_r^2 I_3) \in \mathbb{R}^3, \quad (5)$$

$$\delta q = \exp(\omega) = \left(\cos \frac{\|\omega\|}{2}, \sin \frac{\|\omega\|}{2} \hat{\omega} \right), \quad \hat{\omega} = \omega / \|\omega\|, \quad (6)$$

$$q'_k = \delta q \otimes q_k \quad (\text{quaternion multiplication}), \quad (7)$$

with δq taken to be the identity $(1, 0, 0, 0)$ when $\omega = 0$ (removable singularity of the formula at $\|\omega\| = 0$). Because δq and q_k are both unit quaternions, q'_k is unit norm to machine precision by construction (a defensive renormalization is still applied to absorb floating-point drift over many steps).

Proposition 1 (Symmetry / detailed balance of the SO(3) proposal). *The proposal kernel $q(\cdot | q_k)$ induced by Eq. (7) is symmetric with respect to the Haar measure on $SO(3)$: $q(q'_k | q_k) = q(q_k | q'_k)$.*

Proof sketch. $\mathcal{N}(0, \sigma_r^2 I_3)$ is symmetric under $\omega \mapsto -\omega$, and $\exp(-\omega) = \exp(\omega)^{-1} = \delta q^{-1}$ (the conjugate quaternion), so δq and δq^{-1} are equally likely draws. Left-multiplication by a fixed rotation is measure-preserving for the Haar measure on $SO(3)$ (a standard group-invariance fact), so the map $q_k \mapsto \delta q \otimes q_k$ pushes the (rotation-invariant) proposal distribution over δq into a distribution over q'_k whose density at q_k , having arrived from q'_k via δq^{-1} , equals the density of arriving at q'_k from q_k via δq — i.e. $q(q'_k | q_k) = q(q_k | q'_k)$. This is the same construction used for random-walk proposals on Lie groups in the computational statistics literature (perturb in the tangent space at the identity, transport by the group action). $\square$

Remark 1 (Double cover). q and $-q$ represent the same rotation. The scoring function only ever sees $R(q)$ through Eq. (1), which is invariant to this sign, so no special handling is required for correctness; we simply do not attempt to canonicalize the sign of q'_k , since doing so would not change $\log \pi$.

5.3 Composite, mode-masked proposal

`planning.md` requires the user to choose, at sampling time, which degrees of freedom move: rigid rotations only, rigid translations only, rigid rotation+translation, flexible beads only, or all of the above. This is implemented as a single proposal function parameterized by a boolean mode mask $(\text{rot}, \text{trans}, \text{bead}) \in \{0, 1\}^3$: each excluded component's step is replaced with the identity ($\sigma \rightarrow 0$), which trivially preserves detailed balance (a deterministic identity map is its own reverse). Concretely, per MH step, `proposals.py` draws independent noise for every rigid body and bead (vectorized over K and M with `jax.vmap`) and masks out the disabled groups *before* composing/adding, so the same compiled proposal function serves every mode without retracing.

6 Metropolis correctness

For a symmetric proposal $q(\theta' | \theta) = q(\theta | \theta')$, the Metropolis acceptance probability

$$\alpha(\theta \rightarrow \theta') = \min \left(1, \exp \left[\log \pi(\theta') - \log \pi(\theta) \right] \right) \quad (8)$$

satisfies detailed balance $\pi(\theta) q(\theta'|\theta) \alpha(\theta \rightarrow \theta') = \pi(\theta') q(\theta|\theta') \alpha(\theta' \rightarrow \theta)$ by the standard Metropolis argument. Section 5.2 establishes symmetry for the rotation component; translation and bead components are symmetric Gaussian steps; the masked/identity components are trivially symmetric. The product proposal over independent rigid bodies and beads is therefore symmetric as a whole, so Eq. (8) applies directly with *no* Hastings ratio term – matching the `blackjax.rmh(logdensity_fn, proposal)` contract already documented in `rmh_sampler.py`.

7 Wrapper pipeline

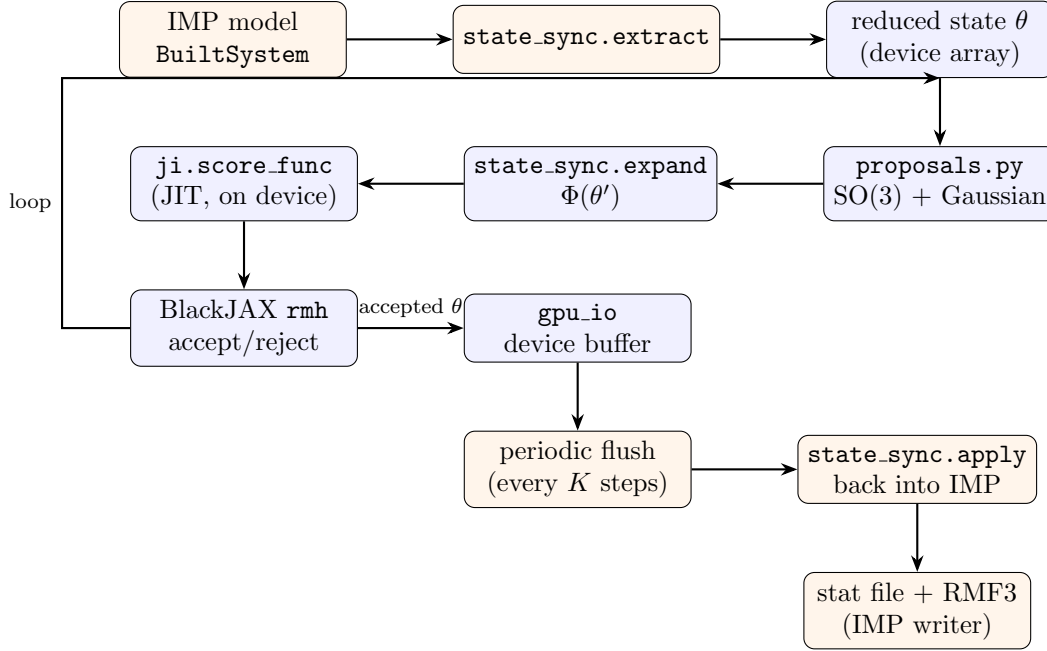


Figure 1: Wrapper data flow. Blue boxes stay resident on device across many MCMC steps (no host round-trip); orange boxes are host/CPU-side and IMP-aware. The MH loop only touches the host every K steps, at the `gpu.io` flush boundary (Section 8).

Algorithm 1 `wrapper_impjax.run_sampling` (single-mode RMH; SMC-tempered variant reuses the same $\log \pi$ /proposal and calls `smc_base_sampler.py` instead)

- 1: `built` $\leftarrow$ `BuiltSystem` (already constructed by the user’s setup script)
 - 2: `ji` $\leftarrow$ `sf.get_jax()`; `layout` $\leftarrow$ `dof_layout.build(built)`
 - 3: `template, r` $\leftarrow$ copy of `ji.get_jax_model()` $\triangleright$ one-time, breaks aliasing (Assumption 1.1)
 - 4: $\theta_0 \leftarrow \text{state_sync.extract}(\text{built}, \text{layout})$
 - 5: $\log \pi(\theta) \leftarrow -\text{ji.score_func}(\{\text{xyz} : \text{state_sync.expand}(\theta, \text{layout}, \text{template}), r\})$
 - 6: `proposal` $\leftarrow$ `proposals.build_composite(layout,  $\sigma_r, \sigma_t, \sigma_b$ , mode)`
 - 7: `run rmh_sampler.run_rmh_sampling(log  $\pi, \theta_0$ , proposal, ...)` $\triangleright$ unmodified existing sampler code
 - 8: every K accepted steps: `gpu.io.flush` $\rightarrow$ `state_sync.apply` $\rightarrow$ write stat row + RMF3 frame
-

8 GPU-aware I/O

Two competing constraints: RMF3 writing is IMP-native and CPU-only (no GPU path exists or is being built — `planning.md` explicitly forbids reimplementing infrastructure that already works), while constant device→host synchronization to write every step would dominate runtime on GPU. The design:

1. Run K MCMC steps entirely on-device inside a single `jax.lax.scan`, accumulating θ_t (or just accepted-state deltas) into a device-resident buffer of shape $(K, 7K+3M)$.
2. Call `jax.block_until_ready` once per block and transfer the whole buffer to host in one copy (batched, not per-step) — this is the same batching discipline already used for scoring in `smc_base_sampler.batched_vmap`.
3. On the host, replay each flushed θ_t through `state_sync.apply` (writes rigid-body reference frames and bead coordinates back into the live IMP model) and append one row to the stat file and one frame to the open RMF3 handle.
4. The RMF3/stat writer stays open across the whole run (one file handle, incremental frame writes) rather than reopening per block.

K (the flush interval) is a tunable knob trading host-transfer overhead against peak device memory for the buffer — exactly the same batch-size trade-off already exposed as `score_batch_size` in `smc_base_sampler.py`, so `gpu_io.py` reuses that naming convention for consistency.

Status (added 2026-08-14): step 1 above (true on-device `jax.lax.scan` accumulation across a block before a single host transfer) is not yet implemented in `custom_rmh.py` or `smc_fixed_schedule.py` — both currently pull every saved sample to host individually inside a Python loop, then flush the whole already-host-side list through `gpu_io.write_block` in one batched pass at the end of the run (steps 3-4 above, which *are* implemented). This is a tracked follow-up, not a design change.

9 Fixed-schedule SMC

A second sampler sits alongside the RMH path of Section 7: Sequential Monte Carlo with a *fixed*, user-specified temperature ladder and *no* adaptive tuning of step sizes or schedule spacing — the “base” SMC sampler, as opposed to a tuned/adaptive-tempered variant (reserved for a future, separate file, `smc_adaptive_tempered.py`; the two must never share a file so one can be swapped or deleted without touching the other). Implemented in `smc_fixed_schedule.py`, driven from the wrapper via `wrapper_impjax.run_smc_sampling`.

9.1 Particle population and tempered target

SMC maintains a population of N independent copies of the reduced state, $\{\theta^{(1)}, \dots, \theta^{(N)}\}$, each an ordinary $\theta = (\{q_k, t_k\}, \{x_m\})$ from Section 2, batched as a pytree with a leading N axis on every leaf. The target at temperature $\lambda \in [0, 1]$ is

$$\pi_\lambda(\theta) \propto p_0(\theta) L(\theta)^\lambda, \quad \log \pi_\lambda(\theta) = \log p_0(\theta) + \lambda \log L(\theta),$$

where $L(\theta) = \exp(-\text{score_func}(\Phi(\theta)))$ is the bare IMP-derived likelihood term (Eq. (3) without any prior mixed in) and p_0 is the same optional prior as Eq. (3). Because `build_log_prob`’s `log_prob_fn` already folds the prior in, `wrapper_impjax.py` exposes a separate `log_likelihood_fn` (bare $\log L$) for this tempering split, so a supplied prior is not double-counted between the $\lambda = 0$ term and the λ -weighted likelihood term.

9.2 Initial population

With no informative prior sampler available (the toy models here default to $p_0 \equiv 1$), the initial population is built by replicating the current IMP-extracted state N times and applying one independent draw of the $\text{SO}(3)$ -aware proposal (Section 5) to each copy, so the population starts spread out rather than identical:

$$\theta_0^{(i)} = \text{proposal}(\text{key}_i, \theta_{\text{IMP}}), \quad i = 1, \dots, N.$$

9.3 Algorithm

Standard SMC (Del Moral et al., 2006), reusing `smc_base_sampler.py`’s schedule functions (`SCHEDULE_REGISTRY`: linear/geometric/sigmoid, generic over array shape, imported directly, not reimplemented) and `blackjax.smc.base` for the actual reweight/resample/mutate step – no SMC algorithm is reimplemented. At each step t (schedule value λ_t):

1. **Reweight:** $w_t(\theta) = (\lambda_t - \lambda_{t-1}) \log L(\theta)$.
2. **Resample:** systematic resampling (`blackjax.smc.resampling.systematic`), unmodified.
3. **Mutate:** n_{mcmc} sweeps of the same symmetric proposal used for RMH, targeting π_{λ_t} , via `blackjax.rmh.build_kernel()` vmapped over the N particles – the same no-Hastings-correction argument of Section 6/Proposition 1 applies unchanged, since the mutation kernel is identical to Section 5’s.

`blackjax.smc.base.init/step` accept an arbitrary pytree population directly (particle count inferred from the leading dimension of the first leaf) – verified against the installed BlackJAX before writing this module, not assumed – so no flattening of θ is needed here either, consistent with Section 2. Only `smc_base_sampler.py`’s own `batched_vmap` scoring helper turned out *not* to be reusable (it assumes a plain $(N, \text{n_dims})$ array); `smc_fixed_schedule.py` defines a small local pytree-native equivalent rather than editing that file.

9.4 Best-particle-only output

Unlike RMH, where every saved sample is written out, SMC output is deliberately pruned: at each temperature step (including the initial one) the single best-scoring particle, $\theta_t^* = \arg \max_i \log \pi(\theta_t^{(i)})$ (full, $\lambda = 1$ log-posterior, for reporting), is recorded; the other $N - 1$ particles at that step are discarded. `run_smc_sampling` writes exactly this sequence $\{\theta_0^*, \dots, \theta_T^*\}$ through `gpu_io.write_block` – one RMF3 frame per temperature step – so the trajectory shows the best model’s progression through the anneal. This mirrors IMP’s own `IMP.pmi.macros.ReplicaExchange` with its `number_of_best_scoring_models` set to 1, which is the natural comparison baseline (Section 12).

9.5 Untuned defaults

Parameter	Default (<code>smc_fixed_schedule.py</code>)
N (particles)	100
Temperature steps	20
Schedule	linear
MCMC sweeps per step	10

These are deliberately modest, fixed values – no adaptive selection – so a comparison run finishes in reasonable time; the adaptive-tempered variant (future, separate file) is where per-run tuning belongs.

10 DEBUG-mode score verification

IMP’s CPU scoring implementation is the existing, extensively tested ground truth; the JAX export (Section 3) is the newer implementation being verified against it, never the reverse. When `debug=True` (`run_sampling/run_smc_sampling`), every `debug_every` steps (RMH) or temperature steps (SMC – typically far fewer of them, so the default is every step there) the current θ is synced into the live IMP model (`state_sync.apply`) and its CPU score compared against the JAX score already used for sampling (recovered as $-\log \pi_{\text{JAX}}$, undoing Eq. (3)’s sign), via `score_verification.ScoreComparisonWriter`. Every checkpoint is appended to a CSV (`step`, `jax_score`, `imp_score`, `abs_diff`); a mismatch above 10^{-3} (absolute) is logged as a warning. Checking is throttled, not per-step, because a CPU `evaluate()` call is comparatively expensive – checking every step would erase the point of sampling on JAX/GPU in the first place.

Empirical result (2026-08-14): on the connectivity-only toy system of Section 3, JAX and CPU-IMP scores agree to floating-point precision at every checkpoint (measured $|\Delta| = 0$). On the KCOIL/ECOIL example system, which additionally includes an excluded-volume restraint, IMP itself emits "No JAX support for close pairs, so using all possible pairs; this may be slow", and a small but consistent absolute mismatch appears (≈ 137 against scores of order 10^6 , i.e. $\sim 0.0035\%$ relative, stable across many steps and both samplers). This is flagged as a localized, IMP-side JAX-support gap for that specific restraint type – not a defect in this repository’s expansion map Φ or state-sync machinery, which is exactly what Section 4’s own regression test already isolates and confirms (perturbing only non-sampled rows does not move the score; the discrepancy tracks the restraint type, not the sync code).

11 Timing methodology

All wall/CPU timing in this package goes through `timing.py`’s `TimingSnapshot` (`start_timing/elapsed_timing`), which pairs `time.perf_counter()` (monotonic, unaffected by system clock adjustments – the standard choice for wall-clock benchmarking in Python) with `time.process_time()` (CPU time, excludes time spent blocked/sleeping). `custom_rmh.py` and `smc.fixed_schedule.py` both report both figures at completion; the sampler-comparison example (Section 12) uses the same utility around each sampler call rather than ad hoc `time.time()`.

12 Sampler comparison

`examples/run_kcoil_ecoil_sampling.py` runs and times any combination of three samplers on the KCOIL/ECOIL system: BlackJAX RMH (`run_sampling`), BlackJAX fixed-schedule SMC (`run_smc_sampling`), and IMP’s own native `IMP.pmi.macros.ReplicaExchange` on an identically-scored system, as the ground-truth-*implementation* baseline (distinct from the ground-truth-*score* role IMP’s CPU scorer plays in Section 10 – here the whole sampler, not just the score function, is IMP’s). Each selected sampler is given its own freshly built system: sampling mutates the live IMP model in place, so reusing one system across samplers would bias whichever ran second. A comparison table (sampler, wall time, CPU time, final/best score) is printed and appended to the shared run log.

Assumption 4 (Cross-sampler start states are not bit-identical). *IMP’s `shuffle_configuration` (its own C++ RNG) and BlackJAX’s `jax.random.PRNGKey` (a separate PRNG system) cannot be seeded into agreement; each sampler’s freshly-built system is independently randomized with the same parameters, not the same starting coordinates. Reported timings and final scores should be read as characteristic of each sampler’s behavior on this system, not as a controlled single-trajectory comparison.*

Empirical result (2026-08-14, toy-scale smoke test): on a single-copy KCOIL/ECOIL system with deliberately small step counts, IMP’s native replica exchange was faster in wall time than either BlackJAX sampler (0.94s/100 frames vs. 2.31s/100 RMH steps and 10.70s for a 15-particle, 6-temperature-step SMC run). This is expected at this scale: JIT compilation overhead is not yet amortized over enough steps, and `smc_fixed_schedule.py`’s per-temperature-step retracing of `update_fn/weight_fn` (inherited from `smc_base_sampler.py`’s identical pattern) adds further constant overhead per step. Whether BlackJAX overtakes IMP’s native sampler is expected to depend on step count, system size, and GPU availability – exactly what this comparison script exists to measure, not to assume.

13 Module layout

File	Key functions	Depends on
<code>dof_layout.py</code>	<code>build</code> , <code>flatten</code> , <code>unflatten</code> , <code>mode_mask</code>	<code>system_info.BuiltSystem</code>
<code>state_sync.py</code>	<code>extract</code> , <code>expand</code> , <code>apply</code>	<code>dof_layout</code>
<code>proposals.py</code>	<code>rotation_step</code> , <code>translation_step</code> , <code>build_composite</code>	<code>dof_layout</code>
<code>custom_rmh.py</code>	<code>run_custom_proposal_rmh</code>	<code>blackjax.rmh</code> , <code>timing</code>
<code>smc_fixed_schedule.py</code>	<code>run_fixed_schedule_smc</code>	<code>smc_base_sampler</code> (<code>schedules</code> only), <code>blackjax.smc</code> , <code>timing</code>
<code>score_verification.py</code>	<code>ScoreComparisonWriter</code> , <code>make_debug_callback</code>	<code>state_sync</code>
<code>logging_config.py</code>	<code>configure_logging</code>	<code>stdlib logging</code> only
<code>gpu_io.py</code>	<code>TrajectoryWriter</code> , <code>write_block</code>	<code>state_sync</code>
<code>wrapper_impjax.py</code>	<code>build_log_prob</code> , <code>run_sampling</code> , <code>run_smc_sampling</code>	all of the above

Each file ships with a focused `test/test_{module}.py` (per the repo’s non-negotiable testing rule), runnable in the `imp` conda environment, which is the only environment in this repo with both IMP and BlackJAX installed together.

14 Assumptions and limitations

- Assumption 3 (frozen auxiliary rows) is scoped to the restraints exercised by `test/test_imp_system.py` and `test/test_systeminfo.py` today (connectivity, excluded volume). A regression test asserts that perturbing a non-sampled row does not change `score_func`’s output for the current restraint set; this test must be revisited whenever a new restraint type (e.g. a multi-resolution EM or crosslink restraint reading a coarser bead) is added.

- Nuisance parameters (copy number, calibration constants, etc., mentioned in `planning.md`) are out of scope for this document; the reduced state θ and $\log p_0(\theta)$ in Eq. (3) are designed to be extended with an additional pytree branch for them without changing the rigid-body/bead machinery.
- *Update, 2026-08-14:* the fixed-schedule SMC variant is no longer deferred – see Section 9. Convergence diagnostics (BlackJAX’s built-in `effective_sample_size/potential_scale_reduction`) remain deferred to a separate, later, self-contained module per `planning.md`, as does the *adaptive*-tempered SMC variant (Section 9’s reserved, not-yet-written `smc_adaptive_tempered.py`).
- The excluded-volume-related JAX/CPU score mismatch reported in Section 10 ($\sim 0.0035\%$ relative) is unresolved and IMP-side (its own warning names the cause: no JAX support for close pairs, so it falls back to all-pairs). Re-run the debug check in Section 10 if IMP’s JAX support for that restraint changes.
- `smc_fixed_schedule.py` redefines (and therefore retraces) `update_fn/weight_fn` as fresh JIT closures every temperature step, inherited unchanged from `smc_base_sampler.py`’s identical pattern. This is a real, measured performance cost at toy scale (Section 12) and a candidate optimization (e.g. closing over λ as a traced argument instead of a Python closure constant) if SMC needs to beat RMH/IMP-native timings on larger systems.